

CO2 AT1 APPLICATION BASED QUESTIONS

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a Deterministic Finite Automaton (DFA) over the alphabet $\Sigma = \{0,1\}$ to validate passwords that end with the pattern "01". Further, minimize the DFA and analyze how the optimized design improves system efficiency in real-world hardware implementations

Aim

To design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ that accepts all binary strings ending with the pattern "01", minimize the DFA, and analyze how the minimized DFA improves efficiency in real-world digital hardware implementations.

Problem Statement

Design a DFA that accepts all binary strings over $\Sigma = \{0,1\}$ such that the **last two symbols are "01"**.

Examples:

Accepted Strings

- 01
- 101
- 1101
- 0001
- 111101

Rejected Strings

- 0
- 1
- 10
- 11
- 1011

Step 1: Define the DFA

A DFA is defined as

$$M=(Q,\Sigma,\delta,q_0,F)$$

where

- **Q** = {q₀, q₁, q₂}
- **Σ** = {0,1}
- **Start State** = q₀
- **Final State** = {q₂}

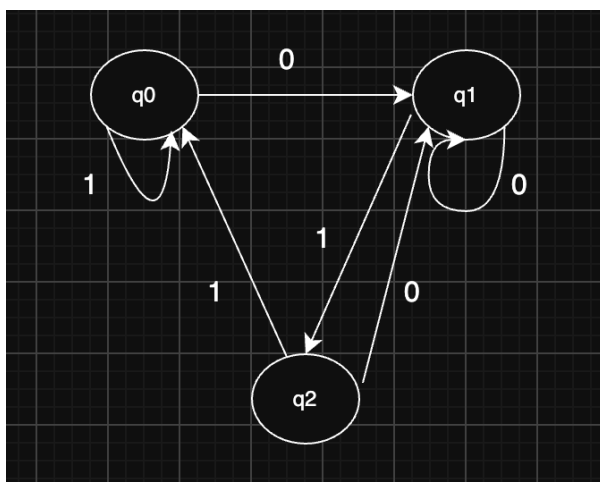
Step 2: State Description

State	Description
q ₀	Initial state; last symbol is not 0
q ₁	Last symbol read is 0
q ₂	Last two symbols are 01 (Accepting State)

Step 3: Transition Table

Present State	Input 0	Input 1
→ q ₀	q ₁	q ₀
q ₁	q ₁	q ₂
*q ₂	q ₁	q ₀

Step 4: State Transition Diagram



Step 5: Working of DFA

Example 1

Input = **1101**

Input	State
Start	q ₀
1	q ₀
1	q ₀
0	q ₁
1	q ₂

Accepted because it ends with **01**.

Example 2

Input = **1011**

Input	State
Start	q ₀
1	q ₀
0	q ₁
1	q ₂
1	q ₀

Rejected because it ends with **11**.

Example 3

Input = **0001**

Input	State
Start	q ₀

0	q ₁
0	q ₁
0	q ₁
1	q ₂ ✓

Result

A DFA over $\Sigma = \{0,1\}$ was successfully designed to accept all binary strings ending with "01". The DFA was minimized and found to be already in its optimal form with **3 states**.

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

1. Introduction

Regular Expressions (Regex) are sequences of characters used to define search patterns. They are widely used in web applications for validating usernames, passwords, email addresses, phone numbers, ZIP codes, and other user inputs before processing them.

2. Alphabet

$$\Sigma = \{a, b\}$$

Required string:

aaaabb

3. Regular Expression

The regular expression is

aaaabb

or

$a\{4\}b\{2\}$

where

- $a\{4\} \rightarrow$ exactly four a's
- $b\{2\} \rightarrow$ exactly two b's

4. Working

Input	Result	Reason
aaaabb	Accepted	Matches exactly
aaabb	Rejected	Only 3 a's
aaaab	Rejected	Only 1 b
aaaabbb	Rejected	Extra b
baaabb	Rejected	Starts with b

5. State Diagram

$(q_0) \xrightarrow{a} (q_1) \xrightarrow{a} (q_2) \xrightarrow{a} (q_3) \xrightarrow{a} (q_4) \xrightarrow{b} (q_5) \xrightarrow{b} ((q_6))$

q_6 = Final State

6. Applications of Regular Expressions

Regular expressions validate:

- Email IDs
- Passwords
- Mobile numbers
- PIN codes
- Usernames
- URLs
- Credit card numbers

7. Advantages

- Fast validation

- Less memory usage
- Prevents invalid input
- Improves security
- Reduces server workload
- Easy to implement

8. Real-World Example

Login forms validate usernames and passwords before sending them to the server.

Example:

Username : john123

Password : Abc@1234

Regex ensures only valid formats are accepted.

Conclusion

Regular expressions provide a simple and efficient mechanism for validating input patterns. Since they describe regular languages, they can be implemented using finite automata, enabling fast and reliable validation in modern web applications.

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a^*b^* can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

1. Introduction

Regular expressions describe regular languages, while finite automata recognize them. Every regular expression has an equivalent finite automaton.

2. Regular Expression

a^*b^*

Meaning:

- Zero or more a's
- Followed by zero or more b's

Examples

Accepted:

ϵ

a

aa

aaa

b

bb

ab

aab

aaabbb

Rejected:

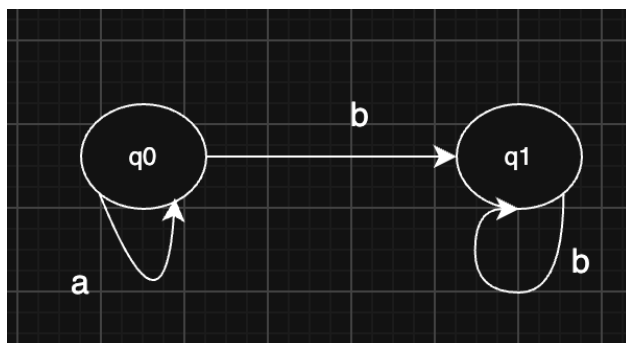
ba

aba

bab

abba

3. Finite Automaton



Any 'a' from q1 → Dead State

4. Transition Table

State	a	b
→*q0	q0	q1
*q1	Dead	q1
Dead	Dead	Dead

5. Why Regex and FA are Equivalent

According to **Kleene's Theorem**:

- Every Regular Expression has an equivalent FA.
- Every FA has an equivalent Regular Expression.

Therefore,

Regular Expression $\Leftrightarrow$ Finite Automaton

Both recognize exactly the **Regular Languages**.

6. Applications

Used in

- Compiler lexical analyzers
- Antivirus software
- Spam filtering
- Intrusion Detection Systems (IDS)
- Log analysis
- Network packet inspection

Conclusion

Regular expressions and finite automata are mathematically equivalent. Their efficient implementation makes them ideal for compilers, intrusion detection systems, and many real-time monitoring applications.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

1. Introduction

The Pumping Lemma is used to prove that certain languages are **not regular**.

2. Pumping Lemma Statement

If L is regular,

there exists a pumping length p

such that every string $|w| \geq p$ can be written as $w = xyz$

where

- $|xy| \leq p$
- $|y| > 0$
- $xy^i z \in L$ for every $i \geq 0$

3. Assume L is Regular

Assume $L = \{ a^n b^n \}$ is regular.

Let pumping length be p

Choose $w = a^p b^p$

Clearly, $|w| = 2p$

4. Divide the String

Since $|xy| \leq p$

both x and y contain only a 's.

Let $y = a^k$ where $k > 0$

5. Pump Down

Take $i = 0$

Then $xz = a^{(p-k)} b^p$

Number of a's $p-k$

Number of b's p

They are not equal.

Therefore, $xz \notin L$

This contradicts the Pumping Lemma.

Hence, L is NOT Regular.

6. Why Finite Automata Cannot Recognize It

Finite automata have finite memory.

They cannot count equal numbers of a's and b's.

Recognizing $a^n b^n$ requires unlimited memory.

7. Compiler Implications

Finite automata are used for:

- Lexical analysis
- Token recognition

They cannot recognize nested structures like $((()))$

`if{while{}} begin...end`

Such patterns require

- Pushdown Automata
- Context-Free Grammars
- Parsing algorithms

Conclusion

The Pumping Lemma proves that $L = \{a^n b^n\}$ is not regular. This limitation explains why compilers use finite automata for lexical analysis and pushdown automata for syntax analysis.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the closure properties of regular languages enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations remains regular, ensuring efficient implementation using finite automata

1. Introduction

Regular languages are closed under several operations, meaning that applying these operations to regular languages always produces another regular language.

2. Closure Properties

If

L_1 and L_2

are regular,

then the following are also regular:

- Union ($\cup$)
- Concatenation ($\cdot$)
- Kleene Star ($*$)
- Intersection ($\cap$)
- Complement ($'$)
- Difference ($-$)
- Reversal

3. Example

Suppose L_1 = strings ending in 01

Regex:

$(0+1)^*01$

L_2 = strings containing 11

Regex:

$(0+1)^*11(0+1)^*$

Union:

$L_1 \cup L_2$

Accepts strings satisfying either rule.

4. Why Result Remains Regular

Regular languages are mathematically proven to be closed under these operations.

Therefore,

Regular + Regular = Regular

Equivalent DFA/NFA can always be constructed.

5. Applications

Closure properties are used in:

- Spam filtering
- Intrusion Detection Systems
- Firewall rule matching
- Email validation
- Search engines
- Lexical analyzers

6. Intrusion Detection Example

Rule 1: admin

Rule 2: password

Combined regex:

admin|password

The IDS detects either keyword using one automaton.

Conclusion

Closure properties allow multiple regular-language rules to be combined while preserving regularity. This guarantees that the resulting language can still be recognized efficiently by finite automata, making regular languages ideal for applications such as intrusion detection, spam filtering, compiler lexical analysis, and user-input validation.